Temporal Subset Approximate Nearest-Neighbor Search

A Workload-Aware Comparison of Global, Partitioned, and Hybrid Strategies

Scott Faust
University of Nebraska Omaha
Omaha, Nebraska, USA
ScottFaust@UNOmaha.com

Abstract

Nearest-neighbor search in high-dimensional data has evolved from exact geometric indexes to approximate nearest-neighbor (ANN) systems that trade exactness for scalability. That literature is now mature for unconstrained similarity search, but many practical systems must answer nearest-neighbor queries over a changing subset of the data rather than over the full collection. This report studies a temporal subset ANN setting in which each record has interval validity, scalar metadata, and optional categorical structure, and each query asks for the top-k nearest neighbors within a time window and filter predicate.

Building on the literature on k-d trees, R-trees, locality-sensitive hashing, product quantization, graph-based search, and large-scale ANN systems, this report develops and evaluates a research prototype with four strategies: an exact filtered oracle, a global ANN plus post-filter baseline, a partition-first ANN method, and a workload-aware hybrid planner. Experiments on synthetic workloads at $n = 5000$ with 32- and 64-dimensional vectors show a clear pattern. Global post-filter ANN performs poorly under subset constraints, with mean Recall@10 ranging from approximately 0.30 to 0.69. Partition-first ANN remains the strongest fixed strategy, achieving Recall@10 of 1.0 across the tested workloads while also producing the lowest median latency. The hybrid planner improves substantially over the global baseline and preserves high recall, but planner match rates of roughly 0.27 to 0.50 and nontrivial latency regret show that adaptive decision quality remains the main unresolved problem. The resulting conclusion is that temporal subset ANN is best treated as a routing and workload-selection problem rather than as ordinary ANN followed by filtering.

Keywords

approximate nearest neighbor, similarity search, temporal filtering, vector search, indexing structures, high-dimensional data

1 Introduction

High-dimensional nearest-neighbor and similarity search is a core problem in algorithms, data structures, and modern intelligent systems. It appears in image retrieval, document search, recommender systems, embedding databases, and many forms of machine learning inference. Classical multidimensional indexes such as the k-d tree and the R-tree established the foundations of geometric search by organizing data so that large portions of the search space could be excluded without direct comparison [2, 7]. As dimensions rise, however, the geometric selectivity that makes exact pruning effective weakens dramatically, and much of the literature shifts toward approximate nearest-neighbor methods that accept bounded error in exchange for practical speed [1, 6, 8].

The literature on ANN is now rich and varied. Hash-based methods use locality-sensitive hashing to preserve similarity probabilistically [6, 8]. Tree-based practical systems such as FLANN use randomized forests and clustered trees, and intrinsic-dimension approaches such as cover trees seek structure beyond the raw ambient dimension [3, 12]. Compression-based methods such as product quantization reduce memory and comparison costs [9]. Graph-based systems such as NSG and HNSW make search itself a traversal problem over approximate neighborhood graphs [4, 11]. GPU-oriented systems further show that large-scale performance depends not only on index structure but also on hardware-aware kernels and memory behavior [10]. Recent work on efficient distance comparison operations suggests that even within established ANN families, the basic cost model of candidate evaluation remains an active systems problem [5].

This report investigates a narrower but practically important variant of the problem: nearest-neighbor search over a *temporal subset*. In this setting, the query does not search the entire vector collection. Instead, it searches only records whose validity interval intersects a requested time window and whose metadata satisfy a scalar constraint. This changes the design problem substantially. A global ANN index can still provide candidate neighbors, but many of those candidates may be invalid under the temporal filter. Conversely, a partitioned search structure can route queries more precisely, but may pay maintenance cost when data expire or become inactive.

The central research question of this report is therefore:

> How should ANN search be organized when nearest-neighbor queries must be answered over dynamic temporal subsets rather than over the full high-dimensional collection?

The report contributes three things. First, it synthesizes the broader ANN literature into a framework that motivates temporal subset search as a natural next problem rather than as a separate topic. Second, it presents a prototype system with four strategies: exact filtered search, global ANN plus post-filtering, partition-first ANN, and a hybrid planner. Third, it evaluates those strategies under multiple temporal workloads and shows that partition-first routing is the strongest fixed method in the current prototype, while adaptive hybrid planning remains promising but incomplete.

2 Literature Review and Related Work

2.1 From exact indexing to approximation

The early literature on multidimensional search was dominated by exact indexing structures. Bentley's k-d tree is one of the clearest examples of this design philosophy: recursively partition the space,

prune subtrees that cannot contain the answer, and rely on geometric structure to reduce query work [2]. Guttman's R-tree extends that same basic intuition to spatial objects and regions rather than only points, using bounding rectangles to organize objects hierarchically for spatial retrieval [7]. These papers are foundational because they establish the central premise that indexing pays off when the geometry of the data allows large parts of the search space to be excluded early.

The key limitation, later emphasized by ANN work, is that this geometric advantage decays in high dimensions. As dimensionality increases, partitions become less selective, bounding regions overlap more heavily, and the number of branches that must be explored rises sharply. In practice, that means exact indexes often drift toward scan-like behavior when used on modern feature vectors or embeddings [1, 6, 8]. This transition is the conceptual bridge between the classical indexing literature and the ANN literature: the question stops being how to answer nearest-neighbor queries exactly at any cost and becomes when exactness is worth preserving.

2.2 The curse of dimensionality as the turning point

The curse of dimensionality is not just a slogan in this literature; it changes the algorithmic regime. Indyk and Motwani explicitly frame approximate nearest neighbors as a response to the fact that decades of effort had still not produced satisfactory exact methods for high-dimensional settings [8]. Gionis, Indyk, and Motwani make the same point in database language, arguing that traditional tree-based methods degrade quickly once dimensionality climbs much beyond the low-dimensional case [6]. The survey by Andoni, Indyk, and Razenshteyn later shows how this observation became the organizing principle for two decades of ANN research [1].

What changes under this view is not merely the objective function but the evaluation criteria themselves. Exact multidimensional indexing is typically justified by asymptotic pruning power. High-dimensional ANN systems, by contrast, are judged by recall, latency, memory footprint, preprocessing cost, update behavior, and robustness to the data distribution [4, 10–12]. This broader perspective matters for the present project because temporal subset ANN is even less about a single asymptotic result and more about a system-level balance among competing costs.

2.3 Hash-based ANN and the first major approximation framework

The first major break from exact geometric search came from locality-sensitive hashing. Indyk and Motwani introduced the formal approximate nearest-neighbor problem and gave locality-sensitive hashing as a way to obtain sublinear behavior in spaces where classical exact indexing was failing [8]. The central insight is that one does not need a partition that perfectly preserves geometry. Instead, one needs hash functions whose collision probabilities are correlated with similarity.

Gionis, Indyk, and Motwani then showed how this approach could be turned into a practical similarity-search mechanism for high-dimensional data, emphasizing that approximate search is often acceptable in applications where the representation itself is already heuristic [6]. Hashing-based methods therefore occupy an important place in the literature for two reasons. First, they provided some of the first strong theoretical intuitions for sublinear ANN. Second, they normalized the idea that approximate answers are often algorithmically and practically preferable to expensive exact search.

At the same time, the later literature makes clear that LSH is not universally dominant. Memory overhead, parameter tuning, and data dependence remain significant issues, and later methods often outperform it on real workloads even if its theoretical framing remains highly influential [4, 9, 12]. That pattern is important for this report because it reinforces the broader theme that no single ANN family solves all workload types.

2.4 Tree-based practical ANN and intrinsic-dimension approaches

Although the literature moved away from exact tree structures, it did not abandon trees entirely. Instead, tree methods were adapted into approximate or data-adaptive forms. Muja and Lowe's FLANN is especially influential because it does not argue for a single dominant structure; rather, it shows that randomized k-d forests and hierarchical k-means trees can each be effective depending on dataset characteristics and desired operating points [12]. This work is important for the present paper because it explicitly frames ANN design as a *selection* problem, not only a data-structure problem.

A different tree-oriented line of work asks whether ambient dimensionality is even the right complexity measure. Cover trees assume that many real datasets have relatively low intrinsic structure even when embedded in high-dimensional spaces. Beygelzimer, Kakade, and Langford show that cover trees can maintain logarithmic-in-n query behavior under bounded expansion assumptions while using linear space [3]. Even if those assumptions do not always hold strongly in practice, the paper is conceptually important because it shifts attention from nominal dimension to underlying structure.

Together, FLANN and cover trees illustrate two different answers to the same question. FLANN accepts heterogeneity and automates method choice across practical tree variants [12]. Cover trees seek a stronger unifying principle based on intrinsic geometry [3]. Temporal subset ANN inherits lessons from both. Like FLANN, it may require strategy selection rather than one universal method. Like cover-tree thinking, it may benefit when constraints shrink the effective search space even if the ambient vector dimension remains unchanged.

2.5 Compression and quantization as indexing strategies

A major development in ANN research was the realization that approximate search could be treated partly as a compression problem. Product quantization is central here. Jégou, Douze, and Schmid show that vectors can be decomposed into subvectors and quantized separately, yielding compact codes that support efficient approximate distance computation [9]. This is a major systems contribution because it changes the resource balance of ANN. Instead of using memory only to store large exact vectors, one stores compressed representations that can still support useful search.

Product quantization also matters because it connects ANN to the realities of very large collections. Once datasets reach millions or billions of vectors, distance computations and storage layout become inseparable concerns. Quantization-based approaches often sacrifice some raw accuracy relative to exact vector comparison, but they make scales feasible that would otherwise be impractical [9, 10]. This systems orientation is directly relevant to the temporal-subset problem because global search over all active vectors becomes less attractive as the number of invalid candidates rises. Compression may help global methods, but it does not solve the subset-routing mismatch by itself.

2.6 Graph-based ANN and the dominance of navigational search

Graph-based methods have become some of the strongest practical ANN approaches because they model search as navigation through an approximate neighborhood graph rather than as bucket lookup or recursive partitioning. Fu et al. explain the appeal clearly: graph search often checks fewer irrelevant points than tree or hashing alternatives because the neighbor relations are represented more directly [4]. Their NSG work is also notable for making explicit that graph quality depends on connectivity, out-degree control, search-path length, and memory usage, not on recall alone [4].

HNSW further strengthens this family by introducing layered small-world structure and logarithmic-style routing intuition. Malkov and Yashunin argue that hierarchical layers separate long-range and short-range links, improving search quality and scalability relative to earlier proximity-graph approaches [11]. HNSW is especially relevant to this project because it represents the ANN backend most closely aligned with the prototype's candidate generation model.

What graph-based ANN contributes conceptually is the shift from *partitioning the space* to *navigating the data*. That is a meaningful distinction for temporal subset ANN. A graph can be excellent at retrieving globally similar items while still being poorly aligned with a subset predicate. The present project's global-filter baseline illustrates exactly that tension: a strong global navigational structure does not necessarily become a strong subset-aware search system once temporal validity is imposed.

2.7 Hardware-aware ANN and the rise of systems concerns

As ANN matured, the literature moved from purely algorithmic comparisons to system-aware optimization. Johnson, Douze, and Jégou show that billion-scale similarity search on GPUs depends not only on the index family but on carefully engineered selection kernels, memory hierarchy use, and compressed-domain computation [10]. In other words, the effective ANN algorithm is no longer only the data structure; it is the joint design of the data structure and the execution platform.

Recent work by Gao and Long pushes that perspective further by focusing on the cost of distance comparison operations themselves. Their argument is that for many modern ANN systems, especially graph-based and quantization-based ones, the dominant cost is often not the high-level index traversal but the repeated distance comparisons used to confirm or reject candidates [5]. This matters for the current project because temporal subset ANN introduces another layer of candidate waste. If many candidates produced by a global ANN strategy are later rejected by subset filters, then the system is paying both traversal cost and distance-comparison cost for ultimately useless work.

2.8 Synthesis: what the literature implies for temporal subset ANN

Taken together, the literature suggests three broad lessons. First, exact geometric indexes remain foundational but do not provide a sufficient solution once dimensionality and workload complexity increase [2, 7, 8]. Second, ANN does not converge to one universally best structure; instead, the field has diversified into hashing, tree-based, quantization-based, graph-based, and hardware-aware approaches whose strengths depend on data distribution, scale, and accuracy requirements [1, 4, 9–12]. Third, modern ANN performance is as much about workload alignment as about raw nearest-neighbor quality.

That third lesson is the most important for this paper. The existing literature largely assumes that the search space is the full vector collection. Temporal subset ANN changes that assumption. The system must solve both similarity search and subset restriction at once. A strong global ANN index may still generate poor end-to-end behavior if many of its best candidates fail the temporal filter. Conversely, a partitioned or hybrid strategy may exploit subset structure even if it is not the globally best ANN method in the ordinary unconstrained setting. This is the gap the present research addresses.

3 Problem Formulation and Research Questions

Each record in the prototype has the form

$$r_i = (v_i, t_i^{start}, t_i^{end}, p_i, c_i),$$

where $v_i \in \mathbb{R}^d$ is the vector embedding, t_i^{start} and t_i^{end} define the record's validity interval, p_i is a scalar price-like attribute, and c_i is an optional category. Open-ended validity is represented by an unset upper endpoint.

Each query has the form

$$q = (x, [T_s, T_e], [P_s, P_e], c, k),$$

where x is the query vector, $[T_s, T_e]$ is the requested time window, $[P_s, P_e]$ is the requested scalar range, c is an optional category restriction, and k is the number of neighbors requested. A record is eligible when its validity interval intersects the query window and its metadata satisfy the scalar and categorical constraints. Defining the eligible subset as

$$S_q = \{r_i : [t_i^{start}, t_i^{end}] \cap [T_s, T_e] \neq \emptyset, \\ p_i \in [P_s, P_e],\ c_i \text{ matches if specified}\},$$

the retrieval goal is to return the top-k nearest records in S_q under Euclidean distance.

This report evaluates three concrete research questions.

RQ1. Does global ANN followed by exact filtering remain competitive once the valid subset becomes small relative to the full collection?

Table 1: High-level comparison of ANN families discussed in the literature review.

Family	Representative work	Main strength	Main limitation for temporal subset ANN
Exact geometric indexing	k-d tree [2], R-tree [7]	Clear pruning logic and strong conceptual foundations in low dimensions	Degrades under high dimensionality; pruning does not directly address dynamic subset predicates
Hash-based ANN	LSH [6, 8]	Strong approximation framework and sublinear-search intuition	Memory and tuning overhead; hash buckets are not inherently aligned with temporal eligibility
Tree-based practical ANN	FLANN [12], cover trees [3]	Practical flexibility and, in some cases, intrinsic-dimension benefits	Performance depends strongly on data characteristics; no guarantee of subset-aware routing
Quantization-based ANN	Product quantization [9]	Excellent compression and scalable approximate distance evaluation	Improves storage and global search cost more than subset selectivity itself
Graph-based ANN	NSG [4], HNSW [11]	Excellent empirical recall-latency tradeoffs and strong navigation behavior	Global graph neighbors may still produce many temporally invalid candidates
Hardware-aware ANN systems	GPU similarity search [10], efficient DCOs [5]	Improves throughput at very large scale and reduces candidate evaluation cost	Optimizes execution efficiency, but does not by itself solve routing under subset constraints

RQ2. Does partition-first routing offer a consistently better fixed tradeoff between recall and latency under temporal subset constraints?

RQ3. Can a workload-aware hybrid planner recover most of the benefit of the best fixed strategy without knowing the workload in advance?

The working hypotheses are correspondingly straightforward: global post-filtering should lose efficiency and sometimes recall under restrictive subsets; partition-first routing should dominate as a fixed strategy when partitions align with query constraints; and a hybrid planner should improve over any single weak baseline, but its quality will depend on the accuracy of subset estimation and strategy selection.

3.1 Why subset constraints change the cost model

The temporal-subset version of nearest-neighbor search is different from unconstrained ANN because the system has two notions of closeness that must be satisfied at the same time. A record can be close in vector space but invalid under the temporal or scalar predicate, or it can be valid under the predicate but far away from the query vector. Ordinary ANN indexes are optimized primarily for the first condition. They are designed to find vector-near candidates quickly, usually without awareness of whether those candidates are eligible for a particular query. Temporal subset search requires both conditions, so a candidate that fails the filter represents wasted index traversal, wasted distance computation, and wasted candidate budget.

This is easiest to see through a simple candidate-budget view. Let a global ANN method retrieve m candidates before filtering. If the selectivity of the predicate is $\sigma_q = |S_q|/n$, then an independence assumption would suggest only about $m\sigma_q$ candidates survive filtering. Real data are rarely independent in this way, but the approximation captures the risk. If the query asks for $k = 10$ results and σ_q is small, the global method may need a much larger candidate budget than it would need for unconstrained ANN. If the implementation caps that budget for latency reasons, recall falls. If it expands the budget aggressively, latency rises. The global method is therefore pushed toward an uncomfortable choice between missing valid neighbors and spending more work on invalid candidates.

Partition-first routing changes the order of operations. Instead of first asking which records are globally close and only later asking which of those records are eligible, the partitioned method first narrows the search to cells that can intersect the query's time and price predicates. The candidate budget is then spent inside a smaller and more relevant region of the collection. This does not make high-dimensional search easy, but it does make the vector search effort better aligned with the actual query semantics. The empirical question is whether that alignment is strong enough to offset the overhead of maintaining partitions and local indexes.

The dynamic nature of the setting adds another cost. A static partitioned design can be tuned once, but temporal validity means records expire, become inactive, or remain open-ended. As a result, the system must manage both retrieval quality and index freshness. Logical deletion and compaction are not secondary implementation details; they are part of the indexing problem. A method that is excellent for a static benchmark may perform poorly if it requires frequent full rebuilds under expiration-heavy workloads. This is why the prototype measures rebuilds, cell rebuilds, compaction counts, deleted counts, and expired counts alongside recall and latency.

These observations motivate the hybrid planner. A global strategy may still be reasonable for broad queries where most records are valid, while exact scanning may be reasonable for tiny subsets where index overhead is larger than the scan itself. Partition-first ANN should be strongest when the subset is selective but not so small that exact scanning is trivial. The planner's role is to estimate which regime a query falls into. The project therefore treats temporal subset ANN as a strategy-selection problem as much as a nearest-neighbor problem.

4 Prototype Design

4.1 Algorithms

The prototype compares four methods.

Exact filtered oracle. The oracle performs a flat scan over the currently active records, applies all filters exactly, computes the exact distances of the surviving candidates, and returns the true top-k. This method is slowest asymptotically, but it provides the correctness baseline and the denominator for recall calculations.

Global ANN then filter. This baseline builds a single ANN index over the active vectors and retrieves approximate nearest neighbors before filtering them by temporal validity and metadata. The method is conceptually simple, but it is expected to degrade when many top ANN candidates fall outside the valid subset.

Partition-first ANN.. The partitioned method routes records by time and price buckets and maintains local ANN indexes inside sufficiently large cells. Small cells fall back to exact local scanning. The method also maintains interval-envelope metadata per cell, including the minimum and maximum interval endpoints, price bounds, open-ended counts, and category counts. These summaries are used to skip cells that cannot intersect the query.

Hybrid planner. The hybrid method estimates subset size from cell metadata and selects among exact search, partition-first ANN, and global ANN plus post-filtering. The current planner is rule-based rather than learned, but it logs its own feature values and outcomes so that planner quality can be analyzed directly.

4.2 Planner features and routing rationale

The hybrid planner is intentionally simple, but its feature set is designed to capture the main signals that determine whether a query should be scanned exactly, routed through partitions, or sent through the global ANN baseline. The most important feature is the estimated subset size. Very small subsets favor exact search because there is little reason to pay index overhead when a short scan can compute the exact answer. Medium-sized selective subsets favor partition-first ANN because the query can avoid most invalid records while still using local ANN behavior. Very broad subsets can make global search more plausible, although the experiments show that this case is still difficult when the candidate budget is too small.

The planner also records the number of candidate cells, the average estimated cell size, the query-window width, the scalar-range width, whether a category filter is present, active-record counts, tombstone counts, tombstone ratio, open-ended interval fraction, and a fragmentation score. These features are not arbitrary. They correspond to different sources of retrieval cost. Large query windows and broad price ranges increase subset size. Category predicates can make otherwise broad queries selective. Tombstone ratios and fragmentation influence whether local structures are clean enough to query efficiently or whether compaction pressure is accumulating. Open-ended intervals matter because they create long-lived records that intersect many time windows and can reduce the selectivity of temporal routing.

Table 2: Planner features logged by the prototype and why they matter.

Feature group	Purpose in planner analysis
Subset-size estimate	Approximates how much exact work remains after applying temporal and metadata filters.
Cell count and average cell size	Indicates whether partition routing will touch a few focused cells or many broad cells.
Query-window and price-range width	Captures predicate breadth before candidate generation.
Category-present flag	Identifies queries with an additional discrete selectivity signal.
Active and tombstoned records	Measures index freshness and the amount of inactive data still visible to structures.
Open-ended fraction	Captures workloads where many intervals remain valid for a long period.
Fragmentation score	Represents how scattered the eligible subset is across routed cells.

This feature design is useful even though the reported planner is rule-based. Because the trace records both the planner's chosen mode and the empirically best mode for each query, the same dataset can be reused for learned planner experiments. In that sense, the rule-based planner is not only a strategy in the evaluation; it is also a data-collection mechanism. Its mistakes identify which feature combinations are not handled well by simple thresholds. The observed low match rates are therefore informative rather than merely disappointing: they show where a future learned planner has room to improve.

4.3 Dynamic maintenance

The prototype supports insertions, logical deletions, and expirations. Records that become inactive are not immediately removed from every structure. Instead, the system uses tombstone ratios and threshold-triggered rebuilds. The global index compacts inactive records during rebuild, and the partitioned index compacts heavily tombstoned cells during local rebuilds. This makes the prototype more realistic than one that rebuilds the full structure after every update.

4.4 Implementation artifact and reproducibility

The implementation is organized as a reproducible Python artifact rather than only as pseudocode. The repository includes separate modules for the exact oracle, global post-filter index, partition-first index, hybrid planner, workload generation, experiment execution, result summarization, plotting, and planner training. The ANN backend is also intentionally optional. When `hnswlib` is installed, the prototype can use an HNSW-backed candidate generator; when it is unavailable, the same interfaces fall back to brute-force candidate generation so that tests, smoke runs, and report-generation scripts remain executable in a minimal environment.

Table 3: Purpose of the workload regimes used in the evaluation.

Workload	Evaluation purpose
Static	Separates pure subset-routing behavior from update and expiration costs.
Short-life expire	Tests frequent expiration, high turnover, and compaction pressure.
Long-life expire	Tests cases where most records remain active but expiration still accumulates.
Open-ended heavy	Tests intervals that remain valid indefinitely and weaken temporal selectivity.

The experiment harness writes per-query CSV traces and run-level summaries. Each query row records latency, Recall@10, valid-result rate, exact subset size, subset-estimate error, planner mode, candidate counts, partitions touched, tombstone counts, deleted and expired counts, rebuild and compaction counts, expansion rounds, and a `best_mode` label. These traces are useful for two reasons. First, they make the reported aggregate figures auditable from query-level observations. Second, they provide the feature and label data needed for learned planner experiments. The artifact therefore includes an initial nearest-centroid planner workflow, although the main results in this report focus on the rule-based hybrid planner.

5 Experimental Methodology

5.1 Synthetic data and workloads

The experiments use synthetic vector data with controllable cluster structure, temporal span, scalar correlation, optional category structure, and interval lifetimes. The final experiments use $n = 5000$ records and evaluate four workload regimes:

(1) **Static**: no turnover.
(2) **Short-life expire**: many records expire quickly.
(3) **Long-life expire**: intervals are long-lived before expiration.
(4) **Open-ended heavy**: many records remain active indefinitely.

Each workload is evaluated in 32 and 64 dimensions. The reported summary statistics aggregate repeated runs over multiple seeds. This setup is intentionally modest enough to keep the artifact reproducible while still stressing the planner and maintenance behavior across materially different temporal regimes.

The four workloads are meant to expose different failure modes rather than simply vary one parameter. The static workload is a control condition. If a method performs poorly there, the cause is not dynamic maintenance but the basic interaction between ANN and subset filtering. The short-life expiration workload is a stress test for maintenance. Many records become invalid quickly, so tombstones and cell rebuilds matter more. The long-life expiration workload is closer to a slowly changing catalog: most records remain available, but the system still needs to handle stale records over time. The open-ended-heavy workload represents applications where many records remain valid indefinitely, such as persistent listings, long-running user profiles, or documents whose availability does not expire. In that setting, temporal filters may be less selective, and the planner must rely more on scalar and category constraints.

Using both 32- and 64-dimensional vectors provides a small but useful dimensionality check. The goal is not to exhaustively benchmark dimensional scaling, but to ensure that the observed temporal-subset behavior is not tied to a single embedding dimension. The results show the same broad pattern in both dimensions: global post-filtering struggles, partition-first routing remains strong, and the hybrid planner improves over the global baseline but does not consistently select the best mode. That consistency makes the main conclusion stronger than it would be under a single-dimensional smoke test.

5.2 Metrics

The evaluation uses:

- Recall@10,
- valid-result rate,
- p50, p95, and p99 latency,
- subset-estimate error,
- rebuild and compaction counts,
- planner mode frequencies,
- planner match rate,
- planner latency regret, and
- planner recall gap.

Planner regret compares the hybrid method to the fastest non-hybrid method that satisfies a recall floor for the same query. This is important because it evaluates the *decision quality* of the hybrid planner, not only the raw retrieval quality.

5.3 Evaluation logic

The oracle defines ground truth. The global and partitioned methods are then evaluated against it per query, which allows recall, latency, and subset-estimation behavior to be studied at query granularity rather than only at the run level. That evaluation design is important because hybrid planners can look strong in aggregate while still making many poor decisions on individual queries. Query-level traces make it possible to isolate exactly that failure mode.

6 Results

6.1 Fixed strategy comparison

The clearest result is that global ANN plus post-filtering is the weakest strategy in this temporal subset setting. Averaged over the two tested dimensions, the global method achieves mean Recall@10 values of roughly 0.31 in the static workload, 0.67 in short-life expire, 0.69 in long-life expire, and 0.67 in open-ended-heavy workloads. In contrast, partition-first ANN maintains Recall@10 of 1.0 in every workload tested. Figure 1 summarizes this pattern.

Latency tells a similarly clear story. Partition-first ANN also produces the lowest median latency across the tested workloads, with average p50 latency between approximately 0.10 ms and 0.39 ms. Exact filtered search is slower but still competitive because the subset constraints reduce candidate counts in many cases. Global ANN plus post-filtering is much slower, often between roughly 1.3 ms and 2.9 ms on average. Figure 2 shows these differences.

Table 4 reports the workload-averaged Recall@10 and p50 latency across dimensions. The strongest fixed conclusion is straightforward: in this prototype, routing search into the relevant temporal

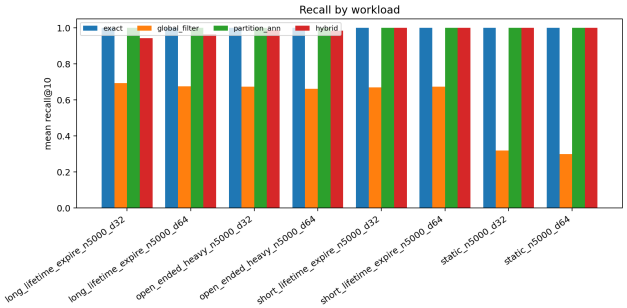

Figure 1: Mean Recall@10 averaged over 32- and 64-dimensional runs at $n = 5000$. Global ANN plus post-filtering degrades substantially under subset constraints, while partition-first ANN remains exact in the tested workloads.

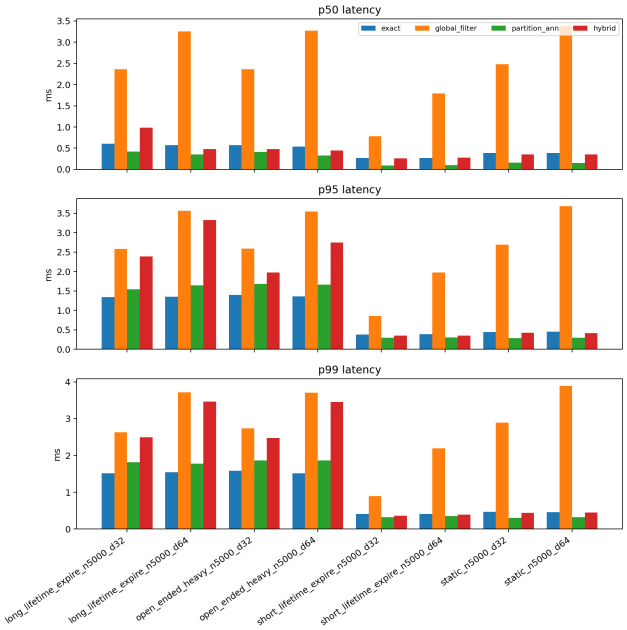

Figure 2: Mean p50 latency averaged over 32- and 64-dimensional runs at $n = 5000$. Partition-first ANN is the fastest fixed strategy in the current prototype.

and scalar partitions is substantially more effective than retrieving neighbors globally and filtering afterward.

6.2 Estimator behavior

A major improvement over earlier versions of the prototype is that subset estimation is no longer the dominant failure point. Mean subset-estimate error is modest across all reported workloads: roughly 0.07 to 0.08 in the static setting, about 0.03 to 0.05 in the short- and long-lifetime settings, and around 0.037 in the open-ended-heavy setting. Figure 3 visualizes these values.

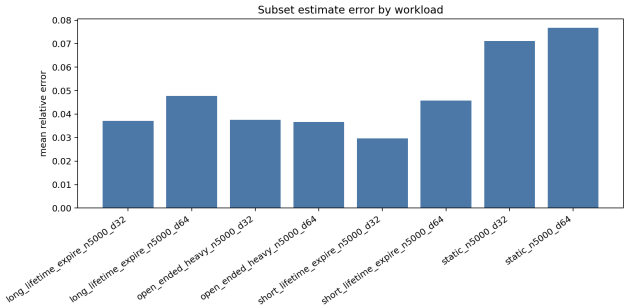

Figure 3: Mean subset-estimate error by workload. Estimation quality is now relatively stable, which shifts attention to planner quality rather than estimator failure.

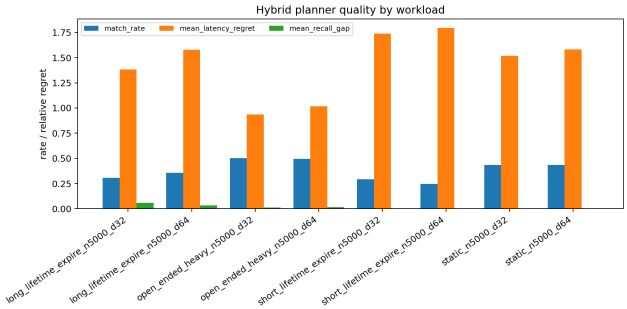

Figure 4: Hybrid planner quality by workload. Match rate remains modest and latency regret remains substantial, showing that planner selection is still the main unresolved problem.

This matters because it changes the interpretation of the hybrid results. Earlier development artifacts suggested that poor subset estimates might explain planner failures under hard temporal regimes. The final artifact indicates otherwise: estimator quality is now reasonably good. The remaining weakness lies in the planner's decision policy.

6.3 Hybrid planner behavior

The hybrid planner substantially improves over the global post-filter baseline. It keeps Recall@10 at 1.0 in the static and short-life workloads, reaches about 0.954 in the long-life workload, and about 0.987 in the open-ended-heavy workload. This is a meaningful improvement over global filtering, especially because valid-result rate remains 1.0 throughout the experiments, which indicates that the loss is due to missed neighbors rather than invalid returned records.

At the same time, the planner is still not choosing the best strategy often enough. Averaged across dimensions, hybrid planner match rate is about 0.43 in the static workload, 0.27 in short-life expire, 0.33 in long-life expire, and 0.50 in open-ended-heavy workloads. Mean planner latency regret ranges from about 0.98 to 1.77. Figure 4 summarizes these planner-quality metrics.

Table 4: Average Recall@10 and p50 latency (ms) across 32- and 64-dimensional runs at $n = 5000$.

Workload	Method	Recall@10	p50 latency (ms)
Static	Exact	1.000	0.389
Static	Global filter	0.309	2.927
Static	Hybrid	1.000	0.354
Static	Partition-first ANN	1.000	0.154
Short-life expire	Exact	1.000	0.269
Short-life expire	Global filter	0.672	1.287
Short-life expire	Hybrid	1.000	0.270
Short-life expire	Partition-first ANN	1.000	0.098
Long-life expire	Exact	1.000	0.585
Long-life expire	Global filter	0.685	2.805
Long-life expire	Hybrid	0.954	0.730
Long-life expire	Partition-first ANN	1.000	0.388
Open-ended heavy	Exact	1.000	0.555
Open-ended heavy	Global filter	0.669	2.816
Open-ended heavy	Hybrid	0.987	0.463
Open-ended heavy	Partition-first ANN	1.000	0.368

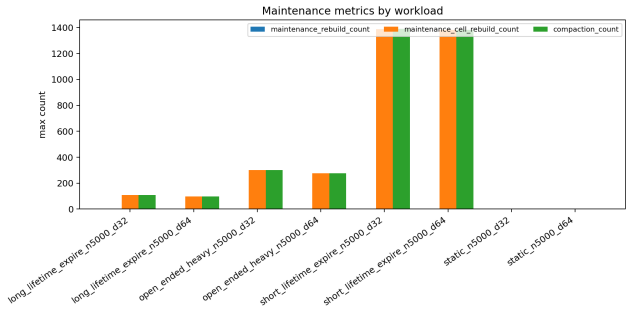

Figure 5: Maintenance-oriented counters by workload. Expiration-heavy regimes increase compaction and rebuild pressure, but the partition-first method still preserves the best retrieval behavior in the current prototype.

This is the most important negative result in the paper. The prototype now demonstrates that *adaptive search is worth attempting*, but it does not yet demonstrate that the current rule-based planner is close to optimal. The planner still overuses exact search in easy settings and overuses global filtering in some hard temporal settings. In other words, the design space is well motivated, but the current planner remains a baseline rather than a final solution.

6.4 Maintenance costs

Dynamic maintenance is visible in the reported rebuild and compaction statistics. Short-life and open-ended workloads generate large cell-rebuild counts and compaction events because those workloads create the heaviest temporal turnover. This is not surprising: the same conditions that make subset routing valuable also increase maintenance pressure. The key point is that the partition-first method still retains its retrieval advantage even under these maintenance-heavy workloads. That suggests the maintenance cost is real but not disqualifying in the current prototype.

7 Discussion

The experiments support three broader conclusions.

First, temporal subset ANN should not be treated as ordinary ANN followed by filtering. The poor recall of the global post-filter baseline shows that the filter changes the problem fundamentally rather than merely narrowing the final answer set. This is consistent with the larger ANN literature, which repeatedly shows that the success of an index depends on how well its structure matches the workload [10–12].

Second, partition-first routing is highly effective in the present prototype. This aligns with the intuition that when queries are defined over changing subsets, search should begin with the subset whenever possible rather than with the full collection. In this report's experiments, the partition-first strategy dominates both exact and global-filter methods as a fixed design choice.

Third, hybridization is promising but incomplete. The hybrid planner clearly improves over the global baseline and often preserves exact-level recall. However, planner match rates and regret show that selecting the right mode remains difficult. The current system therefore supports a nuanced conclusion: *the adaptive formulation is justified, but the planner itself remains the main research challenge.*

A useful way to interpret the fixed-strategy results is through candidate waste. Global ANN retrieval spends effort to find globally similar vectors whether or not they satisfy the temporal and scalar filters. In restrictive workloads, many of those candidates are simply unusable. Partition-first routing reduces that waste by making subset membership part of the routing decision. The prototype results suggest that this structural alignment matters more than squeezing small efficiency gains out of any single ANN kernel.

7.1 Answers to the research questions

For RQ1, the answer is no: global ANN followed by exact filtering does not remain competitive once subset constraints become central to the query. The global baseline is attractive because it is simple and because it resembles the architecture of many vector search

systems: build one index over the collection, retrieve candidates, and then apply application-level predicates. The results show why that architecture is fragile. The global method often spends its candidate budget on records that are close in vector space but invalid under the query. Its valid-result rate remains high because the final filtering step removes invalid records, but its recall drops because too few valid nearest neighbors survive the candidate-generation stage. This is a different failure from returning wrong records; it is a failure to expose enough eligible candidates to the final ranking step.

For RQ2, the answer is much more favorable to partition-first routing. Partition-first ANN is the strongest fixed strategy in the current artifact, producing exact-level Recall@10 in all reported workloads and the lowest median latency among the fixed methods. This does not mean partition-first routing is universally optimal, but it does show that making subset constraints part of the index-routing step is a powerful design choice. The result is especially important because the partitioned method is not merely improving recall at the cost of much higher latency. It improves both recall and median latency in this experiment because it avoids much of the candidate waste that hurts global post-filtering.

For RQ3, the answer is mixed. The hybrid planner improves substantially over the global baseline, and in several workloads it preserves exact-level or near-exact recall. However, the planner's match rate shows that the current rule-based decision policy is not yet reliable enough to be considered a final solution. This distinction matters. A hybrid architecture can be promising even when a particular planner is weak. The prototype demonstrates that the necessary signals are measurable, that planner regret can be computed at query granularity, and that a trace can be produced for training or evaluating future planners. The main unresolved problem is not whether strategy selection is useful, but how to choose the strategy with enough consistency to avoid latency regret.

7.2 Practical implications

The main practical implication is that vector search systems with strong metadata or temporal predicates should be cautious about treating filtering as a final post-processing step. Post-filtering is easy to implement, but it hides a substantial candidate-generation problem. If candidate generation is oblivious to eligibility, the system may need to over-fetch aggressively to recover recall. Over-fetching then increases distance computations and latency, especially in graph-based or quantization-based systems where candidate evaluation is already a major cost.

A second implication is that exact search should not be dismissed automatically. In ordinary high-dimensional ANN discussions, exact scanning is often presented only as a slow baseline. In subset search, exact scanning can be competitive for very small eligible sets because the predicate has already reduced the work. This is one reason a planner is useful. A system that always routes to ANN may do extra work on tiny subsets; a system that always scans may become too slow on larger subsets; and a system that always uses a global index may waste budget on invalid candidates. The best choice changes with query selectivity.

A third implication is that maintenance policy and query policy should be evaluated together. Partition-first routing is valuable partly because it aligns with the query predicates, but the same partitioning creates maintenance obligations. Cells may need local rebuilds, tombstones must be tracked, and open-ended records may intersect many future windows. The prototype results suggest that these costs are manageable at the tested scale, but the maintenance metrics are not incidental. They should be part of any serious comparison of temporal subset ANN designs.

7.3 What should be added next

The most valuable future addition to the paper's experimental story would be a learned planner comparison. The artifact already records the correct ingredients: query features, chosen modes, exact subset sizes, best empirical modes, latency regret, and recall gaps. A learned planner could be trained on held-out seeds and compared against the rule-based planner using the same trace format. This would directly address the current planner weakness without changing the rest of the benchmark harness.

A second useful addition would be a broader scale study. The current report uses $n = 5000$ to keep experiments reproducible and fast enough for repeated development, but larger collections would clarify how maintenance costs and candidate waste grow. In particular, it would be useful to vary n, vector dimension, temporal selectivity, and bucket granularity independently. That would reveal whether partition-first routing remains dominant as local cells grow larger and whether global post-filtering becomes even less attractive at larger scale.

A third useful addition would be a production-style workload or trace. Synthetic data is appropriate for controlled experiments, but real systems often have skewed categories, correlated metadata, bursts of insertions and expirations, and nonuniform query windows. Those properties may change planner behavior. A future version of this study could evaluate whether the same conclusions hold for document retrieval, listing search, recommendation candidates, or event validity windows. That would make the contribution more persuasive to system builders who need to choose an architecture for filtered vector search.

8 Threats to Validity and Future Work

The report has several limitations. The experiments are synthetic rather than based on a production vector database, so real-world distributions may differ. The metadata model includes a single scalar filter and optional category rather than many heterogeneous attributes. The reported hybrid planner is rule-based, and the included nearest-centroid learned-planner workflow is only an initial artifact rather than a robust planner-selection solution. Finally, the current prototype measures dynamic maintenance through rebuild and compaction counts, but it does not yet optimize those operations aggressively.

These limitations suggest several immediate next steps. The most direct is to replace the rule-based planner with a stronger learned planner trained on the logged query features and per-query outcomes. The final artifact already records the information needed for that step, including estimator error, mode choices, planner regret, and workload descriptors. A learned planner could then be evaluated against the rule-based planner using the same benchmark

harness. A second direction is richer subset-aware indexing, including multi-scalar routing, category-aware partitioning, or dual interval indexes over both start and end times. A third direction is systems-oriented optimization. Recent ANN work suggests that faster distance comparison and candidate evaluation can change the relative strength of search strategies [5]; integrating those ideas into the temporal-subset setting may reduce the cost of both global filtering and local exact fallbacks.

9 Conclusion

This report studied temporal subset ANN as an extension of the broader literature on high-dimensional indexing and approximate nearest-neighbor search. The literature shows a progression from exact multidimensional indexes such as k-d trees and R-trees to approximate methods based on hashing, trees, compression, graph navigation, and hardware-aware execution [2, 4, 6–12]. The experimental contribution of this report is to show how those ideas behave once nearest-neighbor queries are constrained to dynamic temporal subsets.

The strongest empirical finding is that global ANN plus post-filtering is a poor fit for this problem, while partition-first ANN is consistently strong in the current prototype. The hybrid planner improves over the global baseline and demonstrates that adaptive strategy selection is worthwhile, but planner match rate and regret show that the current decision policy is still suboptimal. The final conclusion is therefore both practical and forward-looking: temporal subset ANN is best understood as a routing and workload-selection problem, and the next major research step is learning how to choose among otherwise strong search strategies.

References

[1] Alexandr Andoni, Piotr Indyk, and Ilya Razenshteyn. 2018. Approximate Nearest Neighbor Search in High Dimensions. *arXiv preprint arXiv:1806.09823* (2018).

[2] Jon Louis Bentley. 1975. Multidimensional Binary Search Trees Used for Associative Searching. *Commun. ACM* 18, 9 (1975), 509–517.

[3] Alina Beygelzimer, Sham Kakade, and John Langford. 2006. Cover Trees for Nearest Neighbor. In *Proceedings of the 23rd International Conference on Machine Learning*. 97–104.

[4] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search with the Navigating Spreading-out Graph. *Proceedings of the VLDB Endowment* 12, 5 (2019), 461–474.

[5] Jianyang Gao and Cheng Long. 2023. High-Dimensional Approximate Nearest Neighbor Search: with Reliable and Efficient Distance Comparison Operations. *Proceedings of the 2023 International Conference on Management of Data* (2023).

[6] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. 1999. Similarity Search in High Dimensions via Hashing. In *Proceedings of the 25th International Conference on Very Large Data Bases*. 518–529.

[7] Antonin Guttman. 1984. R-Trees: A Dynamic Index Structure for Spatial Searching. In *Proceedings of the 1984 ACM SIGMOD International Conference on Management of Data*. 47–57.

[8] Piotr Indyk and Rajeev Motwani. 1998. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. In *Proceedings of the 30th Annual ACM Symposium on Theory of Computing*. 604–613.

[9] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128.

[10] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547.

[11] Yu. A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836.

[12] Marius Muja and David G. Lowe. 2014. Scalable Nearest Neighbor Algorithms for High Dimensional Data. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 36, 11 (2014), 2227–2240.